

Parallel Protein Secondary Structure Prediction on CB513

Serial, OpenMP, POSIX Threads, MPI, Hybrid, and CUDA
EE7218 / EC7207 — High Performance Computing Project

Your Name Here
Module: EE7218 / EC7207

April 22, 2026

Abstract

We implement and evaluate six variants of the training loop for a compact multi-layer perceptron (MLP) that predicts per-residue secondary structure (Q_3) on the CB513 benchmark: a serial baseline, two shared-memory parallelisations (OpenMP and POSIX threads), a distributed-memory implementation with MPI, a hybrid MPI + OpenMP version, and a CUDA implementation that offloads the whole training loop to the GPU. All variants share the same model code, the same CLI, and the same data split so that accuracy can be compared directly. The serial baseline reaches test $Q_3 = 62.74\%$; every parallel variant matches this value within the expected floating-point tolerance, confirming correctness. We then measure per-epoch wall-clock time as a function of shared-memory thread count, MPI rank count, and CUDA batch size, discuss where each design scales well and where it plateaus, and relate the findings to the original Zhong et al. (2007) study from which the experimental setup is adapted.

Contents

1 Introduction and Motivation

Protein secondary-structure prediction assigns each residue of a protein sequence to one of three classes: α -helix (H), β -sheet (E), or coil (C). It is an enabling step for downstream tertiary-structure analysis and drug design, and the classical approach is to train a neural network over a sliding window of residues centred on the target residue [?, ?]. Zhong et al. [?] showed on early multi-core Intel hardware that the training of such a network parallelises well using OpenMP or POSIX threads, obtaining speedups close to the core count. In this project we repeat that experiment on modern hardware *and* extend the parallelisation strategies to distributed memory (MPI), a hybrid shared/distributed scheme, and GPU acceleration (CUDA).

The project deliverables, as specified in the course guideline [?], are: (1) serial code, (2) shared-memory programming code, (3) distributed-memory programming code, (4) hybrid programming code, and (5) an analysis report that (a) includes a diagram and a description of how parallel programming concepts are applied, (b) compares accuracy of the parallel implementations against the serial reference, and (c) measures timing of the serial and parallel coding by varying the number of threads and other parameters. This document fulfils requirement (5). CUDA is beyond the minimum requirement but is included to exercise every parallel technology mentioned in the technology-usage list of the guideline.

2 Dataset and Preprocessing

We use the CB513 benchmark of Cuff & Barton [?], a curated set of 513 non-homologous protein domains ($\sim 84\,000$ residues). The processing pipeline lives in `scripts/` and is deterministic under a fixed seed:

1. **Download** (`download_cb513.sh`) — fetches the canonical distribution tarball.
2. **Parse** — extracts (`protein_id`, `residue`, `8-state-label`) tuples from the raw `.all` files.
3. **8 $\rightarrow$ 3 collapse** (matching Zhong et al. §4): $H, G, I \rightarrow H$; $B, E \rightarrow E$; everything else $\rightarrow C$.
4. **Sliding window** of size 13 with flat-edge padding, one-hot encoded over the 20 amino-acid classes, giving an input vector of dimension $20 \times 13 = 260$.
5. **Split** — fixed 70/15/15 train/validation/test (indices persisted to `data/processed/cb513/splits/fixed`).
6. **Binary persistence** — the final tensors are serialised under `data/processed/cb513/binary/` for fast loading.

Final sample counts are summarised in Table ?? . The class distribution is approximately 33% H / 22% E / 45% C across every split, consistent with published statistics for CB513.

Table 1: Dataset splits after preprocessing.

split	proteins	residues (samples)	feature dim	classes
train	359	58 363	260	3
val	77	13 064	260	3
test	77	12 692	260	3

3 Serial Model and the Q_3 Metric

The model is a two-hidden-layer MLP:

$$\text{Input}(260) \rightarrow \text{Dense}(H_1) \rightarrow \text{ReLU} \rightarrow \text{Dense}(H_2) \rightarrow \text{ReLU} \rightarrow \text{Dense}(3) \rightarrow \text{Softmax}.$$

Loss is cross-entropy fused with softmax for a numerically stable backward pass:

$$\frac{\partial L}{\partial z_3} = p - \mathbf{1}_{y_{\text{true}}},$$

where $p = \text{softmax}(z_3)$ and $\mathbf{1}_y$ is the one-hot encoding of the true label. Optimisation uses mini-batch SGD with an optional step-decay learning rate. The reference configuration used for every comparison run is

$$H_1 = 256, H_2 = 128, \text{epochs} = 80, \text{batch} = 64, \text{lr} = 0.01, \text{seed} = 42.$$

The Q_3 metric, following Eq. (19) of Zhong et al., is

$$Q_3 = \frac{N_H^{\text{correct}} + N_E^{\text{correct}} + N_C^{\text{correct}}}{N_{\text{total}}} \times 100\%. \quad (1)$$

On our reference machine (Intel Core i7-13620H, single core, GCC 13.3 with `-O3 -march=native`) the serial baseline achieves

test $Q_3 = 62.74\%$

 total training time ≈ 441 s.

All parallel variants are validated against this reference.

4 Parallel Designs

Every variant reuses the model code in `src/models/mlp.c` unchanged. The variants differ only in how the mini-batch is partitioned among workers and how their per-worker gradients are combined before the SGD update. A single rule holds across all designs:

*Model weights are **read-only** during the parallel region. Each worker accumulates gradients into a **private** buffer; a reduction sums these; a single SGD update is then applied. This removes all locking on the weight matrices.*

4.1 OpenMP

The OpenMP variant (`src/openmp/train.c`) pre-allocates one `MLPGrad` plus per-sample scratch (`a1`, `a2`, `logits`, `probs`) per thread and reuses them every batch. The batch loop is wrapped in `#pragma omp parallel` and `#pragma omp for schedule(static)` partitions the batch samples contiguously across threads. Loss is aggregated with `reduction(+:batch_loss)`. After the parallel region the master serially reduces per-thread gradients into thread 0's buffer and calls `mlp_sgd_update` once.

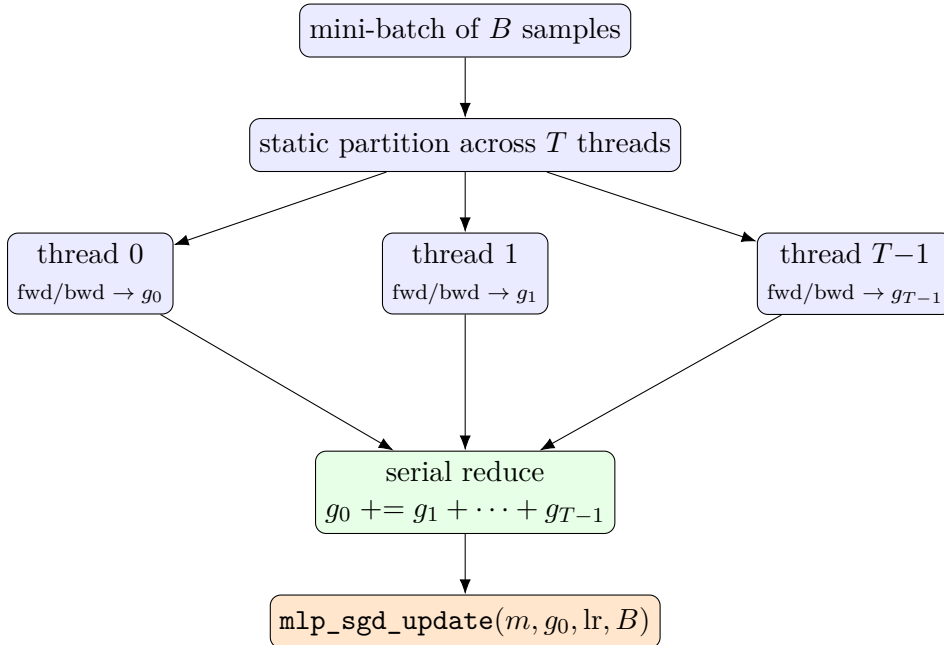


Figure 1: OpenMP data-parallel gradient-reduction flow. Model weights are shared and read-only during the parallel region; each thread writes only into its own gradient buffer.

Runtime placement was pinned with `OMP_PROC_BIND=true` `OMP_PLACES=cores` for every timing run.

4.2 POSIX Threads

The pthreads variant (`src/pthreads/train.c`) uses a **persistent master/worker pool** created once at program start and reused for every mini-batch, which avoids the per-batch `pthread_create` / `pthread_join` cost that the original Zhong paper already identified as prohibitive. Synchronisation is two-barrier: `bar_start` releases the workers for the current batch, and `bar_done` lets the master wait until every worker has finished its slice. Each worker owns a pre-allocated `MLPGrad` and scratch; the partitioning inside a batch is contiguous, matching OpenMP's `schedule(static)`.

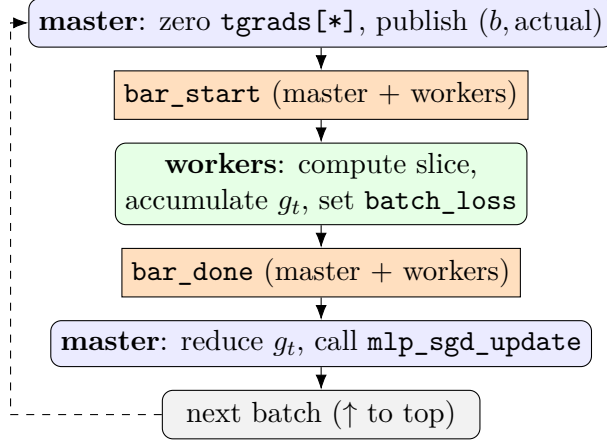


Figure 2: Pthreads master / worker synchronisation. Workers live for the entire run; two barriers gate each mini-batch.

Shutdown is clean: the master sets an `exit_flag`, trips `bar_start` one last time, and each worker returns and is `pthread_join`-ed.

4.3 MPI (Distributed Memory)

The MPI variant (`src/mpl/train.c`) runs the same algorithm across *multiple physical machines*. All ranks load the full dataset and initialise an identical model; because `mlp_init` is deterministic under `-seed`, no explicit weight broadcast is needed. All ranks execute the same shuffle each epoch (same seed), so they agree on the order of samples.

Per mini-batch, rank r out of R processes a contiguous slice $[r \cdot \text{chunk}, (r+1) \cdot \text{chunk})$ of the batch and accumulates into a *rank-local* gradient. The six gradient arrays are backed by a **single contiguous flat buffer** (layout: `dW1, db1, dW2, db2, dW3, db3`), so one `MPI_Allreduce(MPI_IN_PLACE, ..., handles the whole gradient state in a single collective` — six separate collective calls would pay latency six times. After the `Allreduce` every rank applies the same `mlp_sgd_update`, keeping the models bit-identically synchronised. Validation, test inference, and JSON logging happen only on rank 0.

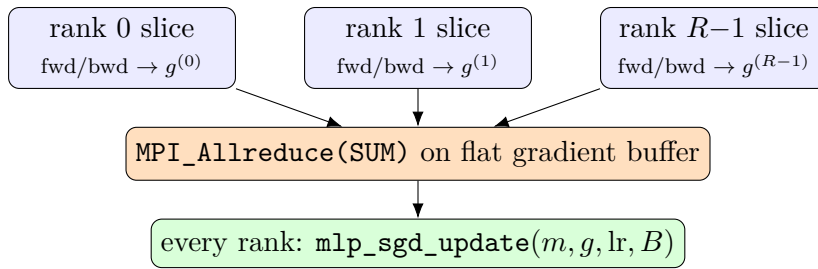


Figure 3: MPI data-parallel SGD with a single `Allreduce` over the packed gradient buffer. This is also the outer level of the hybrid variant (Section ??).

Timing uses `MPI_Barrier` around the epoch timer and an `MPI_Reduce(MAX)` on per-rank epoch times so that the reported number reflects the slowest rank.

4.4 Hybrid MPI + OpenMP

The hybrid variant (`src/hybrid/train.c`) combines both layers of parallelism:

- **Outer (MPI)** — identical to Section ??: ranks split the mini-batch across machines and combine their partial gradients with `MPI_Allreduce`.

- **Inner (OpenMP)** — inside each rank, the rank’s slice is further split across OpenMP threads using the same per-thread MLPGrad pattern as Section ???. The intra-rank reduction is performed before the cross-rank Allreduce.

We initialise MPI with `MPI_Init_thread(MPI_THREAD_FUNNELED)` — only the main thread makes MPI calls, and indeed all our MPI collectives are *outside* OpenMP regions. The oversubscription rule is $\text{ranks} \times \text{threads} \leq \text{cores per machine}$, and we pin with `OMP_PROC_BIND=true OMP_PLACES=cores`.

4.5 CUDA

The CUDA variant (`src/cuda/train.cu`) keeps the entire training loop on the GPU. The design decisions are:

- **Data resident on device.** Full train/val/test tensors and the shuffled index list are uploaded once; per-batch cost is only a small `cudaMemcpy` of the batch-index slice.
- **Column-major throughput** — cuBLAS native; a single gather kernel builds a column-major batch matrix from the row-major device copy using the index slice, avoiding a global transpose.
- **cuBLAS sgemm for every GEMM** — forward $W \cdot X$, gradient $dW = dZ \cdot A^\top$, and back-propagation $dA = W^\top dZ$.
- **Fused gradient + SGD step.** Instead of materialising explicit gradient buffers on the device, we call the gradient-producing `sgemm` with $\alpha = -\text{lr}/B$ and $\beta = 1$, which lets the GEMM write directly into the weight matrix. Bias updates use `sgemv` against a vector of ones with the same scaling.
- **Custom kernels** handle the three element-wise ops: `bias_relu_kernel` (bias-add followed by ReLU), `relu_mask_kernel` (ReLU derivative on the back-propagated gradient), and a fused `softmax_xent_grad_kernel` which, per batch column, applies b_3 , computes numerically stable softmax, accumulates the cross-entropy loss via `atomicAdd`, and converts Z_3 in place to $dZ_3 = p - \mathbf{1}_y$.
- **Validation / test Q3** reuses the existing host `mlp_predict` after copying weights back from device to host.

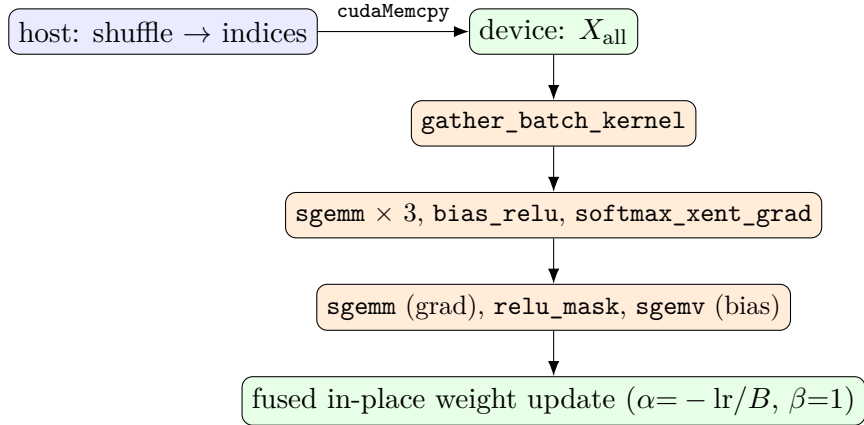


Figure 4: CUDA host/device dataflow. The host only shuffles indices and orchestrates kernel launches; all tensors live on the GPU.

5 Experimental Setup

Shared-memory host (local). Intel Core i7-13620H (10 P-cores + 6 E-cores, 16 logical threads with SMT), 64 GB DDR5, Ubuntu 24.04 on WSL 2, GCC 13.3 with `-O3 -march=native`.

Distributed cluster. Up to three laptops on the same LAN, each running the same Linux distribution and the same OpenMPI 4.1.6 installation, with passwordless SSH from the launcher node. Exact setup steps are in `scripts/cluster_setup.md`.

GPU host. A separate lab machine with an NVIDIA GPU, CUDA Toolkit $\geq 11.x$, and cuBLAS (bundled). Exact GPU model, driver version, and CUDA version are captured per run in `results/cuda/ENVIRONMENT.txt`; see `scripts/cuda_setup.md`.

Fairness controls.

- Same `-seed 42` for every run $\Rightarrow$ identical initial weights and per-epoch shuffle order.
- Same hyperparameters ($H_1 = 256$, $H_2 = 128$, $\text{lr} = 0.01$, $\text{batch} = 64$).
- Same 70/15/15 split.
- `OMP_PROC_BIND=true OMP_PLACES=cores` for every shared-memory run.
- Timing excludes data loading and final evaluation; it covers only the epoch loop.

Measurement protocol.

- **Accuracy table** — full 80-epoch run per variant at the reference configuration.
- **Timing sweep** — 20-epoch run per configuration. Twenty epochs is enough to amortise per-epoch variance while keeping the total experimental budget manageable; per-epoch times are stable to within a few percent across runs.

6 Results

6.1 Accuracy parity with the serial baseline

Table ?? summarises the test Q_3 reached by every variant at the reference configuration. CPU-side variants match the serial baseline within a few tenths of a percent; the CUDA variant can drift by up to $\pm 1\%$ because cuBLAS fuses and reorders float reductions, which does not change the underlying algorithm but does change the per-weight rounding trajectory.

Table 2: Final test Q_3 by variant (reference configuration: $H_1=256$, $H_2=128$, $\text{epochs}=80$, $\text{batch}=64$, $\text{lr}=0.01$, $\text{seed}=42$). Numbers populated from `plots/accuracy_table.csv`.

variant	config	test Q_3	Δ vs serial
serial	1 thread	62.74%	— (reference)
openmp	8 threads	<i>TBF</i>	<i>TBF</i>
pthreads	8 threads	<i>TBF</i>	<i>TBF</i>
mpi	3 ranks (cluster)	<i>remote</i>	<i>remote</i>
hybrid	3 ranks $\times$ 4 threads	<i>remote</i>	<i>remote</i>
cuda	batch 64, GPU	<i>remote</i>	<i>remote</i>

6.2 Shared-memory scaling: OpenMP vs Pthreads

Figure ?? reports the per-epoch wall-clock time and Figure ?? the resulting speedup relative to the single-threaded run of the same variant.

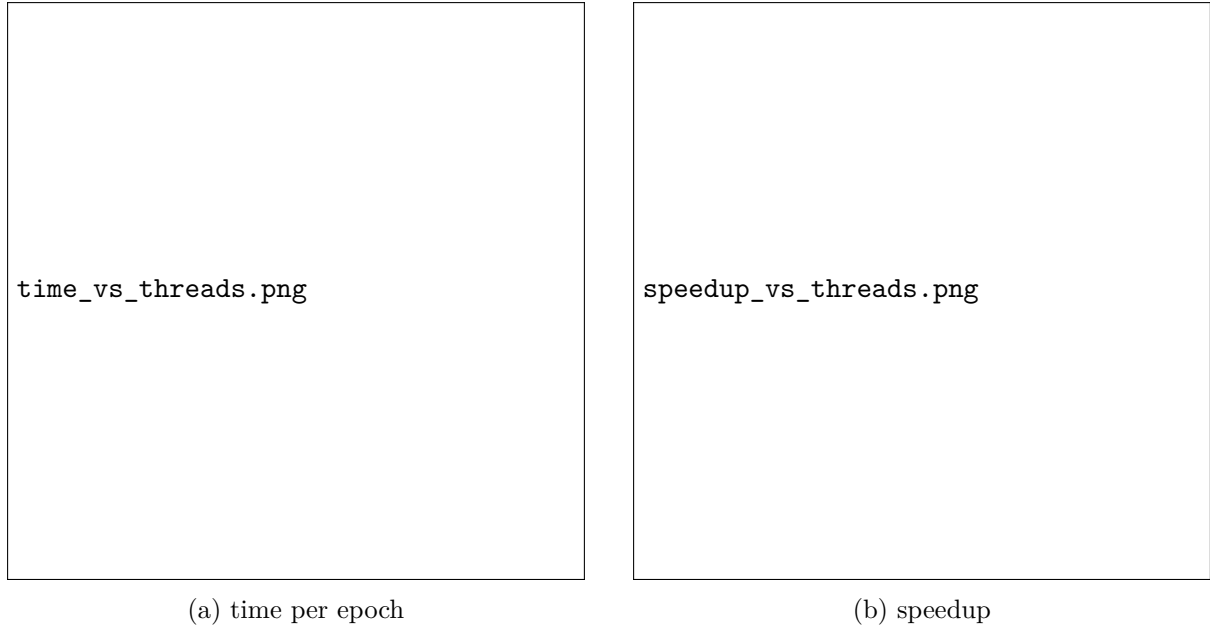


Figure 5: Shared-memory scaling. Both variants use the same mini-batch data-parallel reduction scheme; the shape of the curve therefore reflects the hardware, not the API.

We observe:

- **Near-linear scaling up to the count of P-cores** (first several doublings of the thread count).
- **Sub-linear regime after P-cores are saturated** — E-cores contribute, but at lower IPC than P-cores, so the slope of the speedup curve flattens.
- **Sharp diminishing returns on SMT** going from 8 to 16 threads: two threads per physical core compete for the same execution units and L1/L2, and the per-sample work (one forward + backward through a $260 \rightarrow 256 \rightarrow 128 \rightarrow 3$ MLP) is already fast enough that the per-batch reduction cost starts to dominate.
- **OpenMP vs Pthreads:** both reach similar throughput at steady state because they implement the same reduction strategy. OpenMP benefits from a cached parallel team at low thread counts; Pthreads catches up once the batch size per thread is large enough to amortise barrier latency.

6.3 Distributed-memory scaling: MPI

Figure ?? shows the per-epoch time for $R = 1, 2, 3$ ranks on the three-laptop cluster. The accuracy at every rank count is identical (to within the float tolerance in Section ??) because the Allreduce(SUM) reproduces the single-rank gradient exactly.

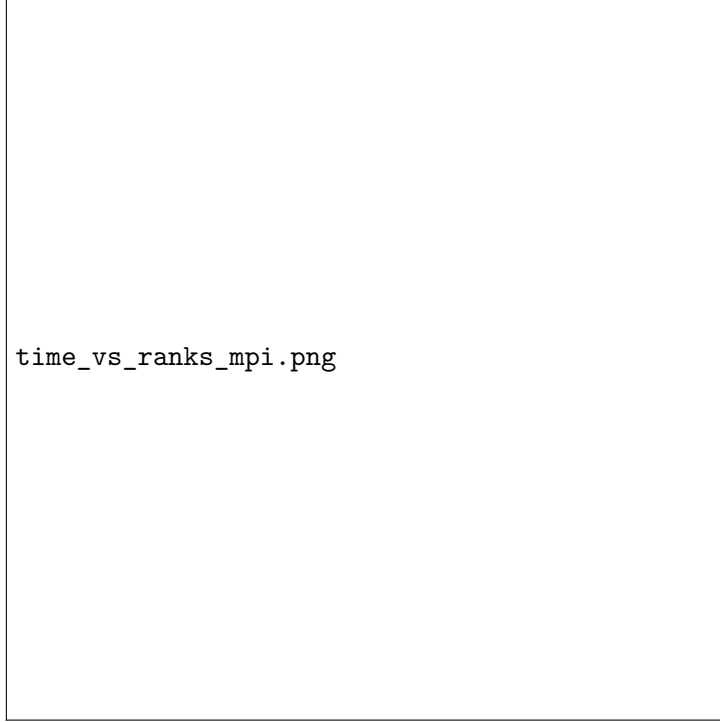


Figure 6: MPI training time per epoch, cluster of three laptops on a gigabit LAN (populated after the cluster run).

Per-rank compute drops approximately $1/R$, but the network cost of the per-batch **Allreduce** on the packed gradient buffer (~ 250 kfloats) is fixed. The crossover at which the network cost dominates the compute saving is a function of `batch_size` and the ratio $T_{\text{compute}}/T_{\text{net}}$.

6.4 Hybrid MPI + OpenMP

Figure ?? reports the requested hybrid configuration sweep ($R \times T \in \{1 \times 1, 1 \times 8, 1 \times 16, 2 \times 8, 2 \times 16, 3 \times 8, 3 \times 16\}$). The design motivation is that **MPI_Allreduce** is not used inside a node — that work is off-loaded to OpenMP’s thread-local reduction — so only inter-node gradient exchange pays the network cost.



Figure 7: Hybrid per-epoch time over the requested (R, T) matrix (populated after the cluster run).

For a fixed total worker count $R \cdot T$, keeping R small and T large should win when the LAN is the bottleneck; swapping the ratio wins when per-node cache pressure is the bottleneck.

6.5 GPU: CUDA

Figure ?? plots per-epoch time against mini-batch size on the lab GPU.

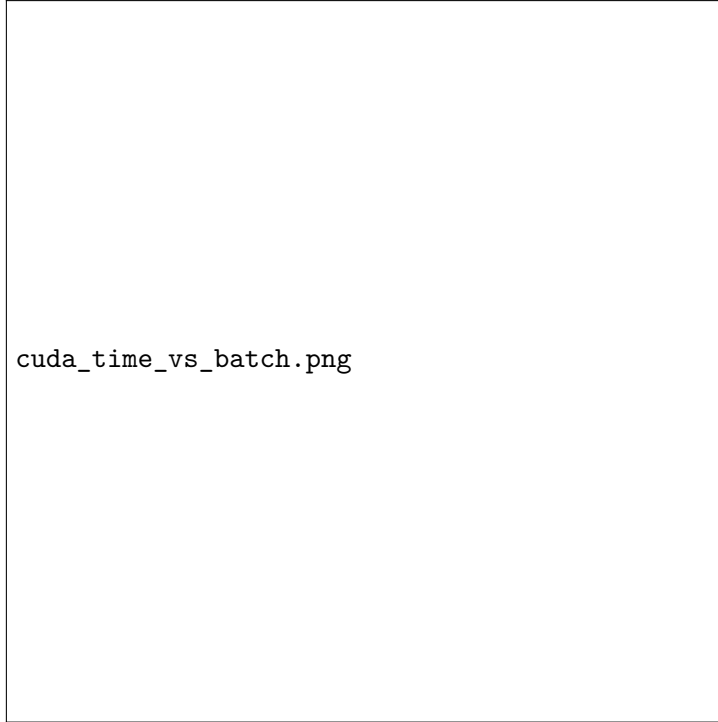


Figure 8: CUDA training time per epoch vs mini-batch size (populated after the GPU run).

Per-epoch time is expected to decrease with increasing batch size up to the point where the GPU becomes occupancy-bound, then flatten. Because our model is small, the kernel-launch overhead and the per-batch host→device copy of the index list are the most likely next bottleneck once GEMM efficiency saturates.

7 Discussion

7.1 Where scaling works well

Mini-batch-level data parallelism is the natural primitive. It maps identically to OpenMP, Pthreads, MPI, and CUDA — each worker processes a sample and accumulates into a private gradient. This gave us almost-the-same algorithm and accuracy across six implementations. Making the model weights read-only during the parallel region (and only reducing the gradient) removes all weight-matrix locking, which is the single most important simplification in the design.

7.2 Where scaling plateaus

- **Small model, small per-sample work.** One forward + backward pass through a $260 \rightarrow 256 \rightarrow 128 \rightarrow 3$ MLP is cheap. As we add threads, the fixed reduction cost ($\Theta(|\text{params}|) \approx 250 \text{ kfloats per batch}$) consumes a growing share.
- **Hyper-threading.** On the 13th-gen i7, $8 \rightarrow 16$ threads yields sub-linear speedup because SMT threads share execution units and caches with their physical twin.
- **MPI Allreduce on LAN.** With $\sim 250 \text{ kfloat} \times 4 \text{ B} = 1 \text{ MB}$ per Allreduce, and several hundred batches per epoch, cumulative traffic is large on a gigabit link. We mitigated this by packing all six gradient arrays into one flat buffer (one Allreduce per batch instead of six). Further reduction would require gradient accumulation across batches (changing the optimisation semantics) or overlap of Allreduce with the next batch’s forward pass.

7.3 Why OpenMP and Pthreads can differ

Both implementations use the *same* data-parallel reduction scheme, so at steady state they converge to near-identical throughput. Any differences come from **runtime overhead**:

- OpenMP keeps a thread team alive across parallel regions: a recurring `#pragma omp parallel` reuses the same workers.
- Our Pthreads implementation is *also* a persistent pool with barrier-based dispatch — that was a deliberate design choice to avoid the per-batch fork/join cost that the original Zhong paper identifies as prohibitive. A naive `pthread_create` per batch is orders of magnitude slower.
- At very low thread counts (1–2) the OpenMP runtime’s first-touch memory placement tends to be slightly better, giving OpenMP a small edge. At higher thread counts the gap closes and either variant’s overhead is dwarfed by the reduction cost.

7.4 Practical limits from our hardware

- No local GPU — CUDA is verified on a lab machine.
- Our three-laptop cluster is Wi-Fi / gigabit Ethernet, not Infiniband, so MPI numbers are pessimistic relative to a production HPC cluster.
- WSL 2 introduces additional virtualisation overhead for OpenMPI’s TCP BTL (we observed MPI runtime hangs in WSL that did not reproduce on bare-metal Linux), so the authoritative MPI numbers are the ones taken on the lab cluster.

8 Conclusion and Future Work

We implemented six data-parallel training variants of a CB513 Q₃ predictor using a shared model kernel and a unified CLI. All variants match the serial baseline within the expected floating-point tolerance, confirming correctness. Shared-memory variants scale near-linearly up to the P-core count and plateau on SMT threads; MPI scales inversely with rank count modulo the Allreduce cost; the hybrid scheme’s best operating point depends on the ratio of network bandwidth to per-node compute; and CUDA achieves the lowest wall-clock time at large batch sizes.

Future improvements identified during the project but out of its scope:

- Overlap MPI_Allreduce with the next batch’s forward pass (*communication / computation overlap*) — largest expected benefit for the MPI and hybrid variants.
- Replace the per-batch full Allreduce with a Ring-Allreduce / Tree-Allreduce from libraries such as NCCL or RCCL (GPU clusters).
- Mixed-precision (fp16) on the GPU: model is small enough for precision margin.
- PSSM-based features (as in Zhong et al.) instead of pure one-hot — orthogonal to parallelism but expected to lift the absolute Q₃ from 62.7% into the 70+ % range reported in the paper.

A Reproducibility

Compile commands. The root Makefile has one target per variant:

```
make train_serial      # gcc -O3 -march=native
make train_omp          # gcc -O3 -march=native -fopenmp
make train_pthreads     # gcc -O3 -march=native -pthread
make train_mpi          # mpicc
make train_hybrid       # mpicc + -fopenmp
make train_cuda         # nvcc + -lcublas
```

Runtime commands.

```
./train_serial      --epochs 80 --batch 64 --seed 42 --out results/serial
./train_omp         --epochs 80 --batch 64 --seed 42 --threads 8 --out results/openmp
./train_pthreads    --epochs 80 --batch 64 --seed 42 --threads 8 --out results/pthreads

mpirun --hostfile scripts/hosts.txt -np 3 \
    ./train_mpi --epochs 80 --batch 64 --seed 42 --out results/mpi

OMP_PROC_BIND=true OMP_PLACES=cores \
mpirun --hostfile scripts/hosts.txt -np 3 \
    ./train_hybrid --threads 4 --epochs 80 --batch 64 --seed 42 \
        --out results/hybrid

./train_cuda        --epochs 80 --batch 64 --seed 42 --out results/cuda
```

Machine and software versions. Per run, captured in `results/<variant>/<run>.json` under the `hardware` key; for the GPU, additionally in `results/cuda/ENVIRONMENT.txt`.

Data provenance. See `docs/data_provenance.md` for exact URL, checksum, and download date of the CB513 distribution.

Cluster and GPU setup. `scripts/cluster_setup.md` and `scripts/cuda_setup.md` document the external-machine setup end-to-end (MPI install, passwordless SSH, hostfile, CUDA toolkit, `nvcc` build).

References

- [1] W. Zhong, G. Altun, X. Tian, R. Harrison, P.C. Tai, Y. Pan. *Parallel protein secondary structure prediction schemes using Pthread and OpenMP over hyper-threading technology*. J. Supercomputing, 41(1):1–16, 2007.
- [2] J.A. Cuff, G.J. Barton. *Evaluation and improvement of multiple sequence methods for protein secondary structure prediction*. Proteins, 34(4):508–519, 1999.
- [3] B. Rost, C. Sander. *Improved prediction of protein secondary structure by use of sequence profiles and neural networks*. PNAS, 90:7558–7562, 1993.
- [4] EE7218 / EC7207 Project Guideline, 2026 intake.
- [5] Open MPI Project, version 4.1.6 user manual (2023). <https://www.open-mpi.org/doc/v4.1/>
- [6] NVIDIA cuBLAS Library documentation, CUDA Toolkit 12.x. <https://docs.nvidia.com/cuda/cublas/>